Z Specification Language Reification, Decomposition & Proof

Transforming Abstract Specifications into Verified Concrete Designs

Lecture Outline

01 **What is Reification?**

Abstract → Concrete transformation

02 **The Retrieve Relation**

Linking two levels of abstraction

03 **Three Proof Obligations**

Init adequacy, Applicability, Correctness

04 **Operation Decomposition**

Refining operations to concrete level

05 **Proof in Z**

Precondition calculation & invariant proofs

06 **Worked Example**

Birthday Book end-to-end

Section 01

What is Reification?

From mathematical abstraction to implementable design

What is Reification?

Reification (also called Data Refinement) is the process of transforming an abstract Z specification into a more concrete design that can be implemented, while formally proving that no correctness is lost.

1

Abstract Specification

Uses sets, relations, functions — mathematically clean, implementation-free

▼ Reification Step ▼

2

Concrete Design

Uses implementable data structures: arrays, lists, records, trees

▼ Reification Step ▼

3

Implementation Code

Actual program in a programming language (Java, C, Python…)

Why is Reification Necessary?

⚡ Abstraction Gap

Z sets and relations are ideal for specification but cannot run directly on a computer. We need implementable structures.

✓✓ Correctness Bridge

Reification provides a formal, mathematical link proving the concrete design correctly implements the abstract spec.

🔍 Design Clarity

Each reification step forces the designer to make deliberate implementation choices and justify them with proofs.

🔒 Safety Guarantee

In safety-critical systems, informal 'it looks right' is not enough. Reification provides a proof certificate.

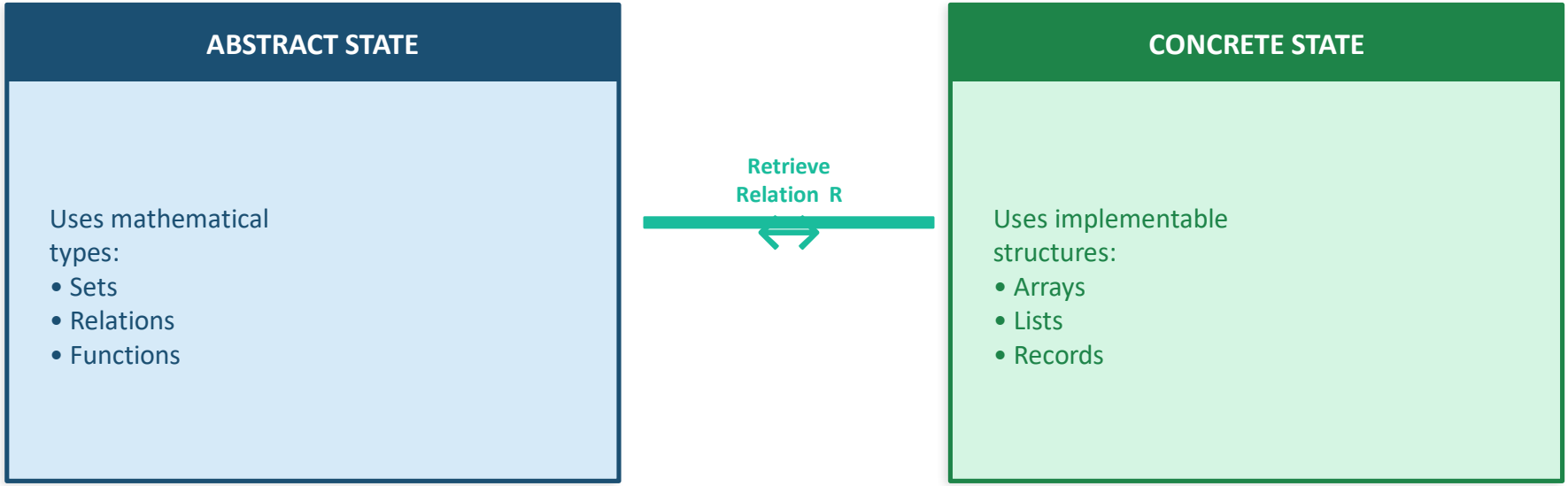
Section 02

The Retrieve Relation

The formal bridge between abstract and concrete states

The Retrieve Relation

The retrieve relation (R) is a formal relation connecting concrete states to the abstract states they represent. It is the mathematical bridge between the two levels.



Key: Each concrete state maps to exactly one abstract state. Multiple concrete states may map to the same abstract state.

Defining the Retrieve Relation — Schema

The retrieve relation is defined as a Z schema that includes both the abstract and concrete state schemas, with a predicate linking them:

```
-- Abstract: birthday stored as relation
AbsBBook
  birthday : PERSON <-> DATE
  known    : P PERSON
  ____________________________
  known = dom birthday
```

```
-- Concrete: stored as a sequence
ConBBook
  bdayList : seq (PERSON x DATE)
  ____________________________
  -- no duplicates on first element
  (functional bdayList)
```

```
-- RETRIEVE RELATION: links concrete to abstract
Retrieve
  AbsBBook    -- brings in: known, birthday
  ConBBook    -- brings in: bdayList
  ____________________________________________________
  birthday = ran bdayList   -- abstract relation = range of concrete list
  known    = dom birthday   -- (inherited from AbsBBook invariant)
```

Section 03

Three Proof Obligations

What must be proved for reification to be valid

The Three Proof Obligations of Reification

To formally verify that a concrete design correctly implements an abstract specification, THREE proof obligations must ALL be discharged:

1

Initialisation Adequacy

Every concrete initial state corresponds to a valid abstract initial state.

$$ConInit \implies \exists\, abs \bullet Retrieve \land AbsInit$$

2

Applicability

Whenever the abstract operation can fire, the concrete operation can also fire.

$$Retrieve \land pre\ AbsOp \implies pre\ ConOp$$

3

Correctness (Result)

After the concrete operation, the result corresponds to a valid abstract result.

$$Retrieve \land pre\ AbsOp \land ConOp \implies \exists\, abs' \bullet Retrieve' \land AbsOp$$

Obligation 1 — Initialisation Adequacy

Goal: Every concrete initial state must correspond to a valid abstract initial state through the retrieve relation.

Formal condition:

```
ConInit  ⟹  ( ∃ abs_state : AbsState • Retrieve ∧ AbsInit )
```

How to prove it:

```
-- ConInit: bdayList = <>              (empty list)
-- AbsInit: known = {}  birthday = {}  (empty relation)

-- Retrieve with these witnesses:
--   birthday = ran <> = {}    ✓
--   known    = dom {} = {}    ✓
-- Therefore ConInit implies a valid AbsInit exists. QED
```

Obligation 2 — Applicability

Goal: The concrete operation must be applicable whenever the abstract operation is applicable (pre-conditions align).

Formal condition:

```
Retrieve  ∧  pre AbsOp  ⟹  pre ConOp
```

Example — AddBirthday:

```
-- pre AbsAdd:  name? ∉ known            (person not already recorded)
-- pre ConAdd:  name? ∉ dom (ran bdayList)  (no entry in list for person)

-- PROOF:
--   Retrieve says: known = dom (ran bdayList)
--   So: name? ∉ known  ⟺  name? ∉ dom (ran bdayList)  ✓
-- Applicability holds.  QED
```

Obligation 3 — Correctness (Result Correspondence)

Goal: After executing the concrete operation, the new concrete state corresponds via Retrieve to a valid abstract after-state.

Formal condition:

```
Retrieve ∧ pre AbsOp ∧ ConOp  ⟹  ∃ abs' • Retrieve' ∧ AbsOp
```

Example — AddBirthday:

```
-- ConAdd: bdayList' = bdayList ^ <(name?, date?)>   (append to list)
-- Need to show: birthday' = ran bdayList'  (Retrieve' holds)

-- PROOF:
--   ran bdayList' = ran (bdayList ^ <(name?, date?)>)
--                 = ran bdayList ∪ {(name?, date?)}
--                 = birthday ∪ {name? ↦ date?}   = birthday'  ✓  QED
```

Section 04

Operation Decomposition

Refining abstract operations to concrete implementations

Operation Decomposition

Operation decomposition refines each abstract operation schema into a corresponding concrete operation schema, using the concrete state representation.

Abstract Operation

```
AbsAdd
  Δ AbsBBook
  name? : PERSON
  date? : DATE
  ________________
  name? ∉ known
  birthday' =
    birthday ∪ {name?↦date?}
  known' = known ∪ {name?}
```

⟹ decompose ⟹

Concrete Operation

```
ConAdd
  Δ ConBBook
  name? : PERSON
  date? : DATE
  ____________
  name? ∉ dom(ran bdayList)
  bdayList' =
    bdayList ^ <(name?,date?)>
```

Decomposition Rule: A concrete operation ConOp decomposes an abstract operation AbsOp if the three proof obligations hold with respect to the retrieve relation.

Steps for Operation Decomposition

1

Identify Abstract Op

Choose the abstract schema to refine (e.g., AddBirthday, FindBirthday, RemoveBirthday)

2

Define Concrete State

Choose implementable data structure (array, list, BST). Write the ConState schema.

3

Write Retrieve

Define the formal retrieve relation connecting ConState variables to AbsState variables.

4

Write Concrete Op

Write the ConOp schema using the concrete state variables. Mirror the abstract behaviour.

5

Prove Applicability

Show: Retrieve ∧ pre AbsOp ⟹ pre ConOp — concrete fires when abstract can.

6

Prove Correctness

Show: Retrieve ∧ pre AbsOp ∧ ConOp ⟹ ∃ abs' • Retrieve' ∧ AbsOp

Section 05

Proof in Z

Precondition calculation and invariant preservation proofs

Types of Proof Obligations in Z

Z specifications generate several types of proof obligations that must be discharged to ensure the specification is sound:

Schema Consistency

Prove the schema predicate has at least one solution — exhibit a witness state satisfying all constraints.

Precondition Calculation

Derive under what conditions an operation may be applied, by existentially quantifying over output and after-state variables.

Invariant Preservation

Prove every operation maintains the state invariant: if the invariant holds before, it holds after.

Refinement Correctness

The three reification proof obligations: initialisation adequacy, applicability, and correctness.

Calculating Preconditions

The precondition of an operation schema is calculated by existentially quantifying over all output variables and after-state (primed) variables:

```
pre Op  ≙  ∃ output!, after_state' • Op
```

Worked Example — pre AddBirthday:

```
pre AddBirthday
  = ∃ birthday', known' •
       (name? ∉ known)
     ∧ (birthday' = birthday ∪ {name? ↦ date?})
     ∧ (known'    = known ∪ {name?})

  = name? ∉ known     ← the existential witnesses exist for any name? ∉ known

RESULT: The precondition is simply: the person is not already known.
```

Proving Invariant Preservation

Invariant of BirthdayBook: known = dom birthday (must hold before AND after every operation)

```
GOAL: Prove AddBirthday preserves  known' = dom birthday'

GIVEN (from AddBirthday schema):
  (1)  birthday' = birthday ∪ {name? ↦ date?}
  (2)  known'    = known ∪ {name?}
  (3)  known     = dom birthday          [invariant holds before]

PROOF (step by step with justification):
  dom birthday'
  = dom (birthday ∪ {name? ↦ date?})     [by (1)]
  = dom birthday ∪ dom {name? ↦ date?}   [dom distributes over ∪]
  = dom birthday ∪ {name?}               [dom of singleton maplet]
  = known ∪ {name?}                      [by (3)]
  = known'                               [by (2)]

∴  known' = dom birthday'   ✓   Invariant is PRESERVED.  QED
```

Proving Schema Consistency

To prove a schema is consistent, we must exhibit a concrete witness — a specific set of values satisfying all predicates in the schema.

General method:

```
To show Schema S is consistent:  find values for all variables such that the predicate holds.
```

Example — BirthdayBook consistency:

```
GOAL: ∃ known : ℙ PERSON ; birthday : PERSON <-> DATE •  known = dom birthday

WITNESS: known = ∅   and   birthday = ∅    (the empty state)

CHECK:   dom ∅ = ∅ = known    ✓

CONCLUSION: BirthdayBook is consistent.  QED
TIP: Always try the trivial empty witness first!
```

Section 06

Worked Example

Birthday Book — full reification end-to-end

Worked Example — Abstract Specification

Step 1: Write the abstract Z specification using mathematical types (sets, relations).

```
-- Basic types
[PERSON, DATE]

-- Abstract State
AbsBBook
  known    : P PERSON
  birthday : PERSON <-> DATE
  ____________________
  known = dom birthday
```

```
-- Init
InitAbs
  AbsBBook
  ____________
  known = {}

-- Invariant: known is exactly
-- the persons with a birthday.
```

```
-- Abstract Add operation
AbsAdd
  Δ AbsBBook ;  name? : PERSON ;  date? : DATE
  ____________________________________________
  name? ∉ known
  birthday' = birthday ∪ {name? ↦ date?}    known' = known ∪ {name?}
```

Worked Example — Concrete Specification

Step 2: Choose implementable data structure. Here: a sequence of (PERSON, DATE) pairs.

```
-- Concrete State
ConBBook
  bdayList : seq (PERSON x DATE)
  ____________________________
  -- all persons distinct in list
  #bdayList = #(dom(ran bdayList))
```

```
-- Concrete Init
InitCon
  ConBBook
  ____________
  bdayList = <>
```

```
-- Concrete Add operation
ConAdd
  Δ ConBBook ;  name? : PERSON ;  date? : DATE
  ____________________________________________
  name? ∉ dom (ran bdayList)           -- person not already in list
  bdayList' = bdayList ^ <(name?, date?)>  -- append pair to end of list
```

Worked Example — Retrieve & Proof Summary

```
-- RETRIEVE RELATION
Retrieve ≙ AbsBBook ∧ ConBBook  |  birthday = ran bdayList
```

1

Obligation 1: Init Adequacy

ConInit (bdayList=<>) ⇸ ran <> = {} = birthday, dom {} = {} = known ✓

2

Obligation 2: Applicability

Retrieve ∧ name?∉known ⇸ name?∉dom(ran bdayList) by Retrieve definition ✓

3

Obligation 3: Correctness

ran(bdayList^<(name?,date?)>) = ran bdayList ∪ {name?↦date?} = birthday' ✓

All three obligations discharged → ConAdd correctly reifies AbsAdd ✓

Key Mathematical Laws for Z Proofs

$$\mathrm{dom}\,(R \cup S) = \mathrm{dom}\,R \cup \mathrm{dom}\,S$$

Domain distributes over union

$$\mathrm{dom}\,\{x \mapsto y\} = \{x\}$$

Domain of a singleton maplet

$$\mathrm{ran}\,(R \cup S) = \mathrm{ran}\,R \cup \mathrm{ran}\,S$$

Range distributes over union

$$\mathrm{ran}\,(s \frown t) = \mathrm{ran}\,s \cup \mathrm{ran}\,t$$

Range of concatenated sequences

$$\mathrm{dom}\,(f \oplus g) = \mathrm{dom}\,f \cup \mathrm{dom}\,g$$

Override domain rule

$$(f \oplus g)(x) = g(x) \quad \text{if } x \in \mathrm{dom}\,g$$

Override: g takes priority

$$S \cup \emptyset = S$$

Empty set identity for union

$$\mathrm{dom}\,(S \lhd R) = S \cap \mathrm{dom}\,R$$

Domain restriction law

Step-by-Step Proof Strategy

Follow this structured approach in every Z proof to ensure rigour and full marks in exams:

1

STATE the goal

Write clearly what must be proved. E.g. known' = dom birthday'

2

LIST the hypotheses

Write every given fact: schema components, invariant (before), preconditions

3

APPLY laws step by step

Use set laws, function laws, definitions. Each step on a new line.

4

JUSTIFY every step

In brackets after each step: cite the hypothesis or law used.

5

CONCLUDE with QED

State the required property is established. Mark it clearly.

Common Mistakes to Avoid

⚠ Forgetting to define Retrieve

Fix: Always write the retrieve relation BEFORE attempting any proof. Every proof obligation depends on it.

⚠ Proving only one obligation

Fix: ALL THREE obligations (init, applicability, correctness) must be proved — not just correctness.

⚠ Skipping steps in proofs

Fix: Every step needs a justification in brackets. Unsupported steps score zero.

⚠ Confusing before and after state

Fix: After-state variables are primed (birthday', known'). Never mix primed and unprimed in the same step.

⚠ Not checking invariant after ops

Fix: Prove known' = dom birthday' for every Δ operation. Without this, the schema is incomplete.

Key Takeaways

Reification	Transforms abstract specs (sets/relations) to concrete designs (arrays/lists). Proves correctness is not lost.
Retrieve Relation	The formal bridge: R maps concrete states to abstract states. Define it FIRST.
3 Obligations	Initialisation adequacy + Applicability + Correctness. ALL must be proved.
Decomposition	Each abstract operation is refined into a concrete operation. Proves using Retrieve.
Z Proof method	State goal → list hypotheses → apply laws step-by-step → justify each → QED.